

EXAM 1 REVIEW

CWID: _____

NAME: _____

- 1.) Using mathematical software such as Python, compute the singular value decomposition of these matrices, and verify that each result satisfies the equation $A = UDV^T$:

$$\text{a. } \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \qquad \text{b. } \begin{bmatrix} 1 & 3 & -2 \\ 2 & 7 & 5 \\ -2 & -3 & 4 \\ 5 & -3 & -2 \end{bmatrix}$$

Write a Python function that creates the diagonal matrix $D = U^T A V$ to check the results (always recommended). One can see the effects of roundoff errors in these calculations, for the off-diagonal elements in D are theoretically zero. (Note: for a function, you will need to boot your function into memory and define these matrices in your command console).

- 2.) Given the data

x_i	1.0	2.0	3.0	4.0	5.0
y_i	1.3	3.5	4.2	5.0	7.0

- a.) Write a Python script that computes the singular value decomposition technique to find the least squares polynomial of degree 1.
 b.) (CHALLENGE!) In applied mathematics, a fancy way of finding the least squares line through a dataset is through something called a *normal equation*. Look up and derive the normal equation:

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

Implement a Python function that computes the least squares polynomial using this method.

- 3.) Experiment with the QR algorithm and the singular value decomposition of matrices using a Python function. Try examples with four types of equations $Ax = b$ —namely, (a) the system has a unique solution; (b) the system has many solutions; (c) the system is

inconsistent but has a unique least-squares solution; (d) the system is inconsistent and has many least-squares solutions.

- 4.) Write a Python script to get the largest/dominant eigenvalue for the given matrix using the Power Method:

$$\begin{bmatrix} 2 & 1 & 2 \\ 1 & 3 & 2 \\ 2 & 4 & 1 \end{bmatrix}$$

- a.) Using the inverse power method to get the smallest eigenvalue and eigenvector for the same matrix, see how many iterations do you need for it to converge to the smallest eigenvalue.
- b.) Write a Python function that implements the QR method to decompose the same matrix, and verify that Q is orthogonal.
- 5.) Write a Python script that implements Householder's method to produce a symmetric tridiagonal matrix that is similar to the given matrix:

$$\begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix}$$

- a.) Write a Python function that implements the Householder's method. Here are your test cases to type into your Spyder console:

a. $\begin{bmatrix} 12 & 10 & 4 \\ 10 & 8 & -5 \\ 4 & -5 & 3 \end{bmatrix}$

b. $\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$

c. $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$

d. $\begin{bmatrix} 4.75 & 2.25 & -0.25 \\ 2.25 & 4.75 & 1.25 \\ -0.25 & 1.25 & 4.75 \end{bmatrix}$

- 6.) The linear system $Ax = b$ given by

$$\begin{aligned} 4x_1 + 3x_2 &= 24, \\ 3x_1 + 4x_2 - x_3 &= 30, \\ -x_2 + 4x_3 &= -24 \end{aligned}$$

- a.) Write a Python script that implements the conjugate gradient method to solve this system.
- b.) Write a Python function that uses this linear system as a test case.

- 7.) Find the first 2 iterations of the SOR method with $\omega = 1.1$ for the following linear systems, using $x(0) = 0$:

a.
$$\begin{aligned} 3x_1 - x_2 + x_3 &= 1, \\ 3x_1 + 6x_2 + 2x_3 &= 0, \\ 3x_1 + 3x_2 + 7x_3 &= 4. \end{aligned}$$

b.
$$\begin{aligned} 10x_1 - x_2 &= 9, \\ -x_1 + 10x_2 - 2x_3 &= 7, \\ -2x_2 + 10x_3 &= 6. \end{aligned}$$

- a.) Write a Python function that uses these systems as a test case.
b.) Write a Python script to solve this system numerically.
- 8.) (CHALLENGE!) Suppose that an object can be at any one of $n+1$ equally spaced points $x_0, x_1, \dots, x_n$. When an object is at location x_i , it is equally likely to move to either x_{i-1} or x_{i+1} and cannot directly move to any other location. Consider the probabilities $\{P_i\}_{i=0}^n$ that an object starting at location x_i will reach the left endpoint x_0 before reaching the right endpoint x_n . Clearly, $P_0 = 1$ and $P_n = 0$. Since the object can move to x_i only from x_{i-1} or x_{i+1} and does so with probability $\frac{1}{2}$ for each of these locations,

$$P_i = \frac{1}{2}P_{i-1} + \frac{1}{2}P_{i+1}, \quad \text{for each } i = 1, 2, \dots, n-1.$$

Where:

$$\begin{bmatrix} 1 & -\frac{1}{2} & 0 & \dots & 0 \\ -\frac{1}{2} & 1 & -\frac{1}{2} & \dots & 0 \\ 0 & -\frac{1}{2} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_{n-1} \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Write three separate Python scripts that solves this system using a) Gauss-Seidel, b) Jacobi and c) SOR method for: $n = 10, 50$, and 100 .

- 9.) Compute a) the eigenvalues and associated eigenvectors and b) the L2 and L-inf norm of the following matrices:

a.
$$\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$$

b.
$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

c.
$$\begin{bmatrix} 0 & \frac{1}{2} \\ \frac{1}{2} & 0 \end{bmatrix}$$

d.
$$\begin{bmatrix} 1 & 1 \\ -2 & -2 \end{bmatrix}$$

Write two separate Python functions for parts A and B using these matrices as test cases.

- 10.) Use appropriate Lagrange interpolating polynomials of degrees 1, 2, and 3 to approximate each of the following:
- $f(8.4)$ if $f(8.1) = 16.94410$, $f(8.3) = 17.56492$, $f(8.6) = 18.50515$, $f(8.7) = 18.82091$
 - $f(-\frac{1}{3})$ if $f(-0.75) = -0.07181250$, $f(-0.5) = -0.02475000$, $f(-0.25) = 0.33493750$, $f(0) = 1.10100000$
 - $f(0.25)$ if $f(0.1) = 0.62049958$, $f(0.2) = -0.28398668$, $f(0.3) = 0.00660095$, $f(0.4) = 0.24842440$
 - $f(0.9)$ if $f(0.6) = -0.17694460$, $f(0.7) = 0.01375227$, $f(0.8) = 0.22363362$, $f(1.0) = 0.65809197$

You are free to write a Python script or a Python function using these data points as test cases but you must symbolically compute the first, second and third degree polynomials and visualize them using Matplotlib.

- 11.) For the given functions $f(x)$, let $x_0 = 0$, $x_1 = 0.6$, and $x_2 = 0.9$. Construct the Lagrange interpolating polynomials of degree (i) at most 1 and (ii) at most 2 to approximate $f(0.45)$, and find the actual error

- $f(x) = \cos x$
- $f(x) = \sqrt{1+x}$
- $f(x) = \ln(x+1)$
- $f(x) = \tan x$

You can use a Python function and use these as test cases or a script to solve this problem directly.

- 12.) (CHALLENGE!) Given the following population table:

Year	1960	1970	1980	1990	2000	2010
Population (thousands)	179,323	203,302	226,542	249,633	281,442	307,746

a.) Write a Python script that finds the Lagrange interpolating polynomial that passes through the years 1950, 1975, and 2020. What is the value of the population at these data points? Visualize the interpolating line of best fit using Matplotlib.

- 13.) Write a script that finds the Lagrange interpolating polynomial that passes through $x_1 = 11$, $x_2 = 13$, $x_3 = 14$, $x_4 = 16$, and $x_5 = 21$. Use $f(x) = x \sin(x)$ to compute the associated y values. (a) Your script should output the polynomial as a symbolic expression and (b) its associated plot as a solid black line in the interval $[10, 25]$ with the data points displayed as large red asterisks.
- 14.) This problem is pulled off from one of the lecture handouts:

Exercise 5: Write a function that finds the number of iterations N that the sequence $\{\mathbf{x}^{(k)}\}$ defined by $\mathbf{x}^{(0)} = \mathbf{x}$ and $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)}/2$. Your input should be the vector $\mathbf{x}$ of any size and a tolerance $TOL > 0$. The algorithm should then divide every element of the original vector $\mathbf{x} = \mathbf{x}^{(0)}$ by the number 2, that is, $\mathbf{x}^{(1)} = \mathbf{x}^{(0)}/2$ followed by $\mathbf{x}^{(2)} = \mathbf{x}^{(1)}/2$, and so on. This will generate the sequence $\{\mathbf{x}^{(k)}\}$ until the stopping criteria $\|\mathbf{x}^{(N)} - \mathbf{x}^{(N-1)}\|_\infty < TOL$ is satisfied. Your output should be the first positive integer N for which the stopping criterion is satisfied.

- 15.) This problem is also pulled off from one of the lecture handouts:

Exercise 6: Write a function for the L_∞ matrix norm. Your input should be the matrix A , and your output should be $\|A\|_\infty$. Do not use `np.linalg.norm`.

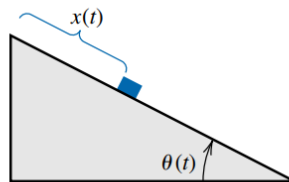
Exercise 7: Write a function for the L_2 matrix norm. Your input should be the matrix A , and your output should be $\|A\|_2$. Do not use `np.linalg.norm`.

- 16.) (CHALLENGE!) The particle in the figure starts at rest on a smooth inclined plane whose angle θ is changing at a constant rate

$$\frac{d\theta}{dt} = \omega < 0.$$

At the end of t seconds, the position of the object is given by the model:

$$x(t) = \frac{g}{2\omega^2} \left(\frac{e^{\omega t} - e^{-\omega t}}{2} - \sin \omega t \right).$$



Suppose the particle has moved 1.7 ft in 1 s. Find, to within $10e-5$ and using a Python script, the rate ω at which θ changes using a) Bisection method, b) Newton's method, and c) Secant method. Assume that $g = -32.17 \text{ ft s}^{-2}$.

- 17.) This problem was given in one of the handouts:

Exercise: Write a script in Python that uses the bisection method to compute p_6 for $f(x) = x^3 + 4x^2 - 10$ starting from $[1, 2]$. (Answer: $p_6 = 1.359375$)

- 18.) Solve problem 17 again using Newton's method.
 19.) Solve problem 17 again using the Secant Method.
 20.) Player A will shut out (win by a score of 21–0) player B in a game of racquetball with probability

$$P = \frac{1+p}{2} \left(\frac{p}{1-p+p^2} \right)^{21},$$

where p denotes the probability that A will win any specific rally (independent of the

server). Determine, to within 10^{-3} , the minimal value of p that will ensure that A will shut out B in at least half the matches they play using:

- a) Bisection method
- b) Newton's method
- c) Secant method

You have the choice to either use a script or a function.